



**Barcelona
Supercomputing
Center**

Centro Nacional de Supercomputación



Applying a Task-Based Approach to Distributed Machine Learning Workflows

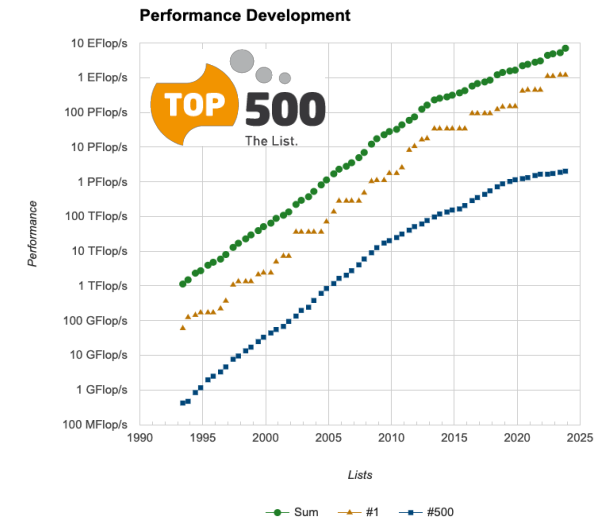
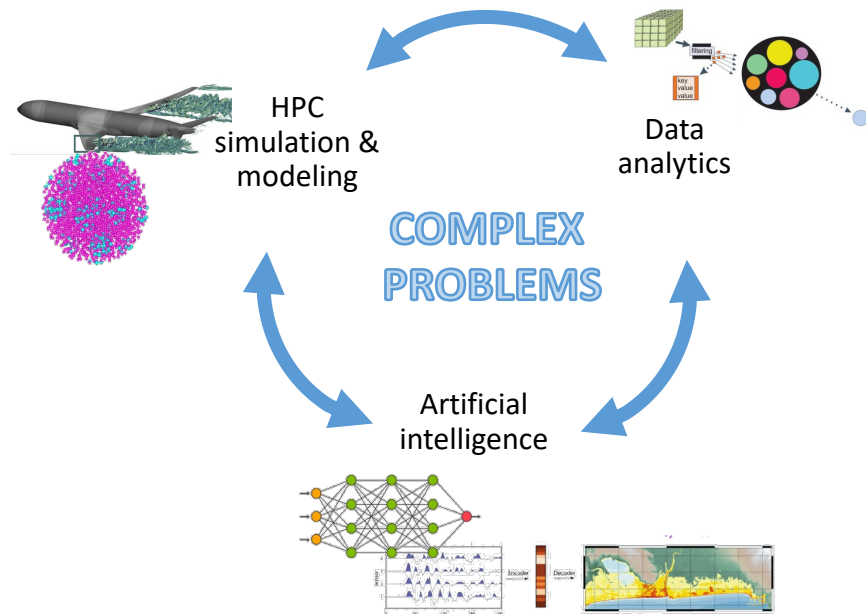
Fernando Vazquez-Novoa, **Daniele Lezzi**, Francesc Lordan, Fatemeh Baghdadi, and Davide Cirillo

17/11/2024

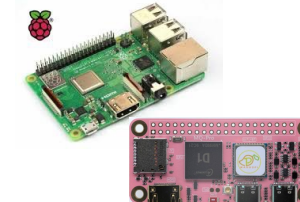
PAW-ATM 2024, In conjunction with SC24 - Atlanta

Complex problems for complex computing infrastructures

- Availability of data continuously generated by sensors, instruments and other devices
- Evolution of computing paradigms from centralized data centers to the **edge** of the network where data must be processed
- **Machine learning** is gaining relevance with the growth of available data to generate more accurate models together with the increment of the computational capacity

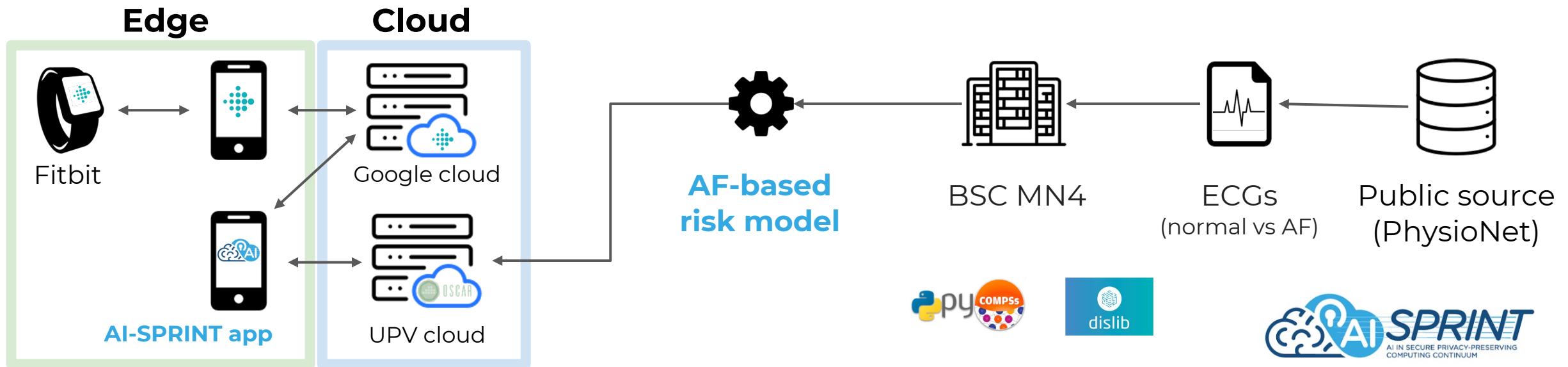


Digital continuum



ML for edge-cloud continuum

- Development of distributed ML models for **cardiovascular monitoring** using portable or wearable devices
 - Training on HPC resources, data collection in the Edge, inference in the Cloud
- Adoption of PyCOMPSs as **programming model for the continuum**
 - Same code runs on multiple backends and orchestration is up to the runtime
- **Distributed ML algorithms** internally parallelized by COMPSs reduce the burden on the developer



Dataset overview and issues



**Barcelona
Supercomputing
Center**

Centro Nacional de Supercomputación

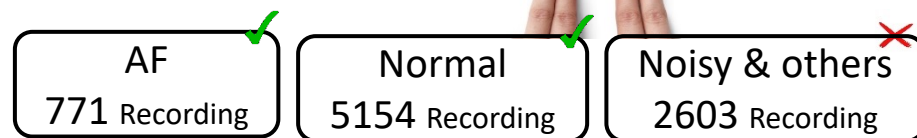
Dataset Overview and Data Preprocessing Steps

Data Source (Public Dataset)



The 2017 PhysioNet/CinC Challenge:

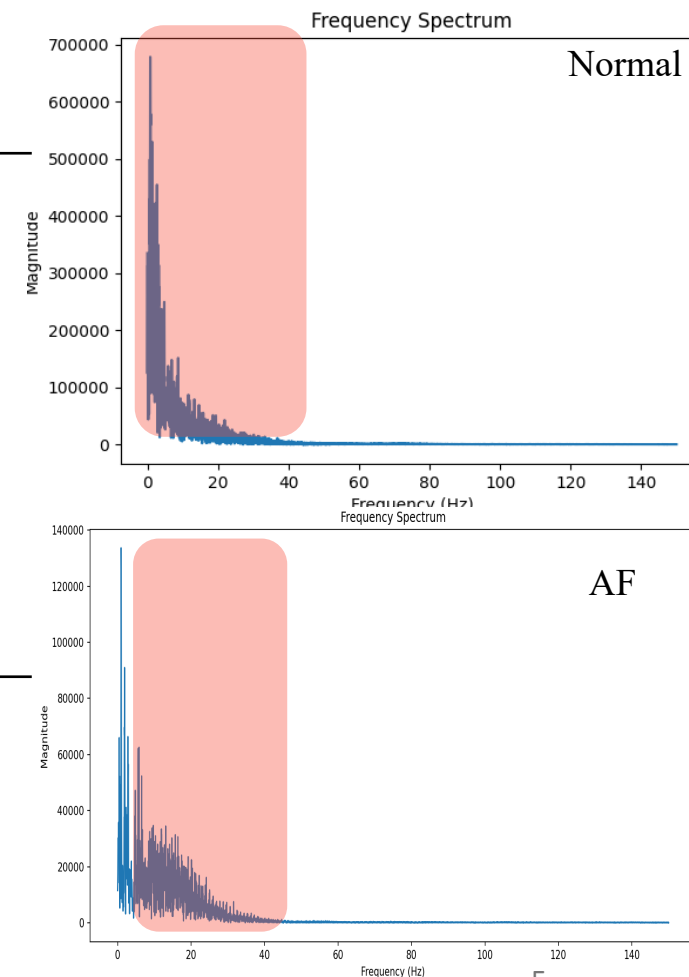
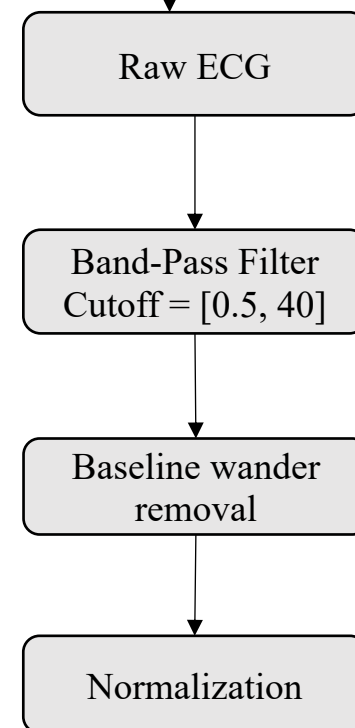
AliveCor single Channel ECG Device, 300Hz sampling frequency, MATLAB V4 WFDB-compliant.



Class imbalance: shuffling-based augmentation
Time differences: zero-padding <30sec & segmentation >30sec



Data Preprocessing

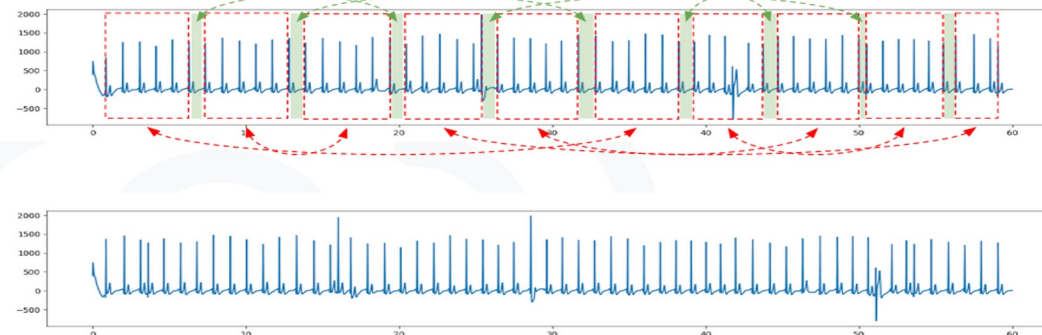


Dataset Preprocessing

- **Class Imbalance:**
 - In medical datasets, certain conditions (like 'abnormal') are often **underrepresented** compared to more common conditions (e.g., 'normal').
 - This imbalance can lead to **biased model predictions**, as models may prioritize the majority class, leading to poor generalization on minority class data. Class imbalance can **reduce model sensitivity** and specificity for underrepresented conditions, limiting clinical utility.
- **Variable-Length ECG Signals for Consistent Model Training:**
 - **Inconsistent Signal Lengths** - non-sustained (less than 30s) or sustained (30s or longer)- have varying durations (between 30s and 60 s in length), which can complicate data processing and model training.
 - Many machine learning models require **fixed-length** inputs for consistent feature extraction and analysis, so variable-length signals need to be standardized.

Synthetic Data Generation with Window Permutation (Window Slicing)

- A technique to create synthetic ECG signals by slicing and permuting sections (windows) of existing ECG recordings.



Zero-Padding for Fixed-Length Signals

- Extending shorter ECG signals with zeros until they reach a fixed length To Maintain Signal Structure and Keep original ECG features intact without altering the physiological data
 1. **Set a fixed length:** that covers most samples or aligns with model input requirements.
 2. **Pad Shorter Signals:** Add zeros at the end of shorter signals to match the target length.
 3. **segments longer signals:** segment longer signals into two segments to match the target length.

The Programming Framework



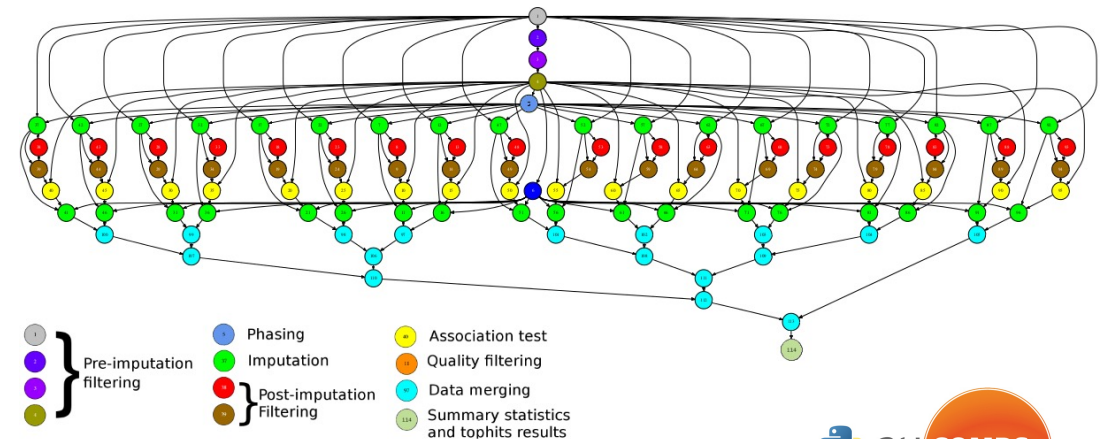
**Barcelona
Supercomputing
Center**

Centro Nacional de Supercomputación

Computational Workflows in PyCOMPSs

- Sequential programming, parallel execution
 - General purpose programming language + annotations/hints
- Task-based parallelization
 - Automatic generation of task graph
 - Coarse grain tasks: methods and web services
 - Sequential and parallel tasks
- Offers a shared memory vision in a distributed system
 - Can address larger dataset than storage space
- Agnostic of computing platform
 - Clusters, clouds and containers cluster
- Based in Python

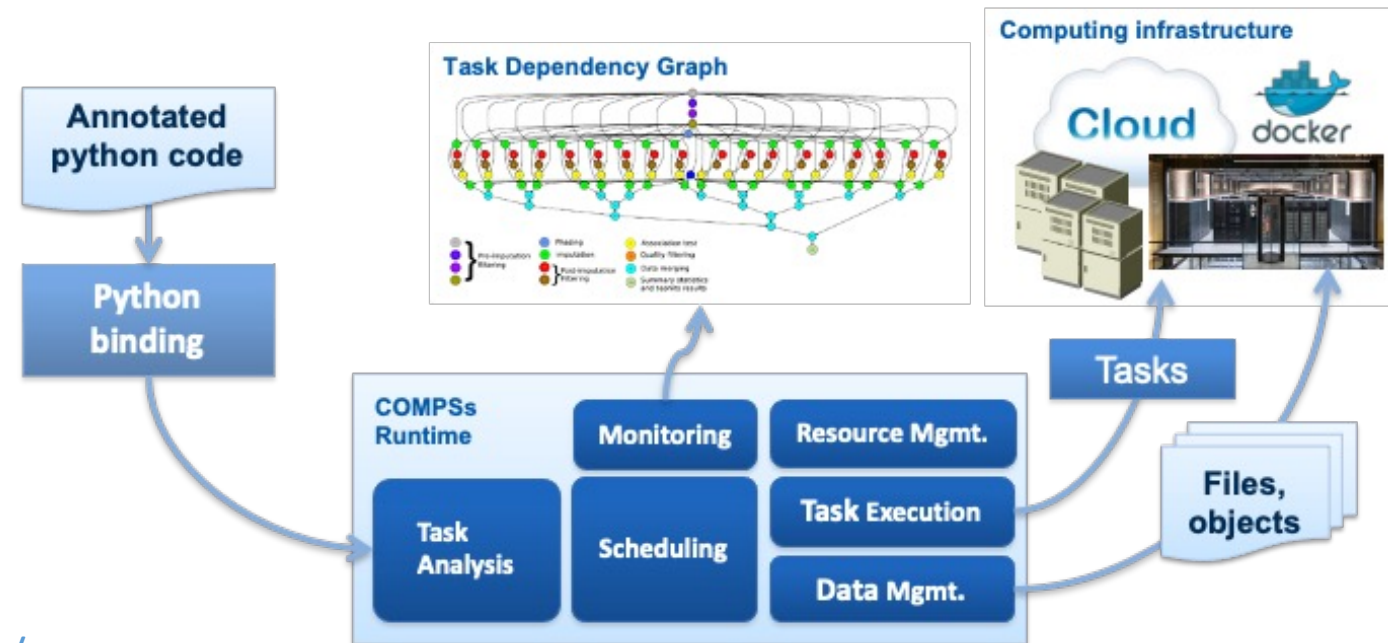
```
@task (c=INOUT)
def multiply(a, b, c):
    c += a*b
```



PyCOMPSs features and runtime

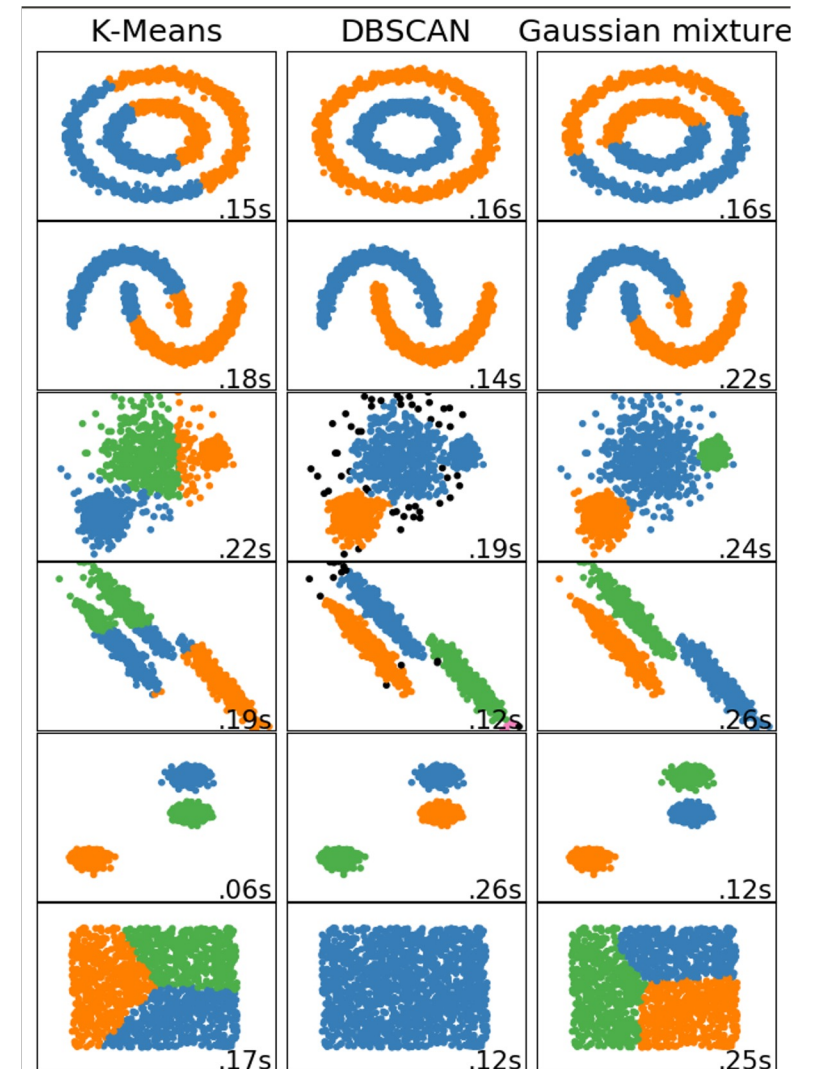
- Support for tasks' constraints – support for heterogeneous infrastructure
- Support for tasks' faults and tasks' exceptions
 - Enlarges the dynamicity of the type of workflows that we support
- Streamed data
 - ... and many others*
- PyCOMPSs applications deployed as a distributed master-worker
 - Executed in an allocation of an HPC system
- All data scheduling decisions and data transfers by the runtime
- Support for horizontal elasticity

*<https://compss-doc.readthedocs.io/>



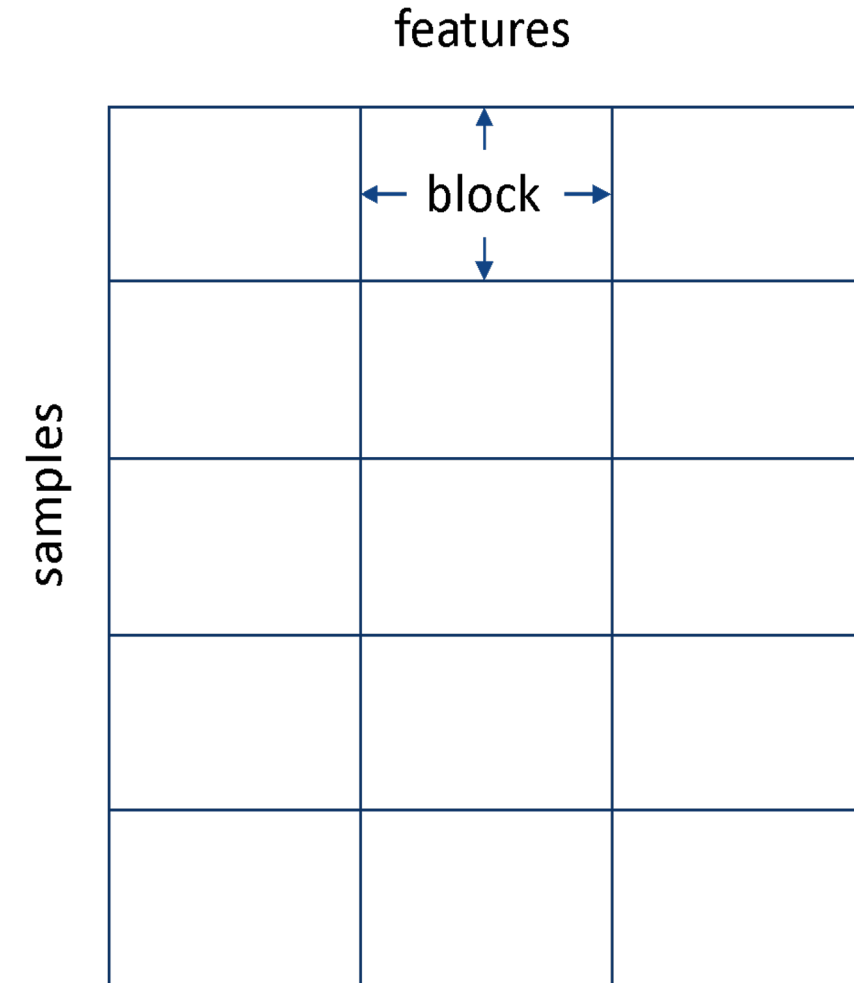
Dislib: parallel machine learning

- dislib: Collection of machine learning algorithms
 - Unified interface, inspired in scikit-learn (fit-predict)
 - Based on a distributed data structure (ds-array)
 - Unified data acquisition methods
 - Parallelism transparent to the user – PyCOMPSs parallelism hidden
 - Open source, available to the community
- Provides multiple methods:
 - data initialization
 - Clustering
 - Classification
 - Model selection, ...



Distributed array (ds-array)

- 2-dimensional structure (i.e., matrix)
 - Divided in blocks (NumPy arrays)
- Works as a regular Python object
 - But not always stored in local memory!
- Methods for instantiation and slicing with the same syntax of numpy arrays:
 - Internally parallelized with PyCOMPSs:
 - Loading data (e.g. from a text file)
 - Indexing (e.g., `x[3]`, `x[5:10]`)
 - Operators (e.g., `x.min()`, `x.transpose()`)
- ds-arrays can be iterated efficiently along both axes
- Samples and labels can be represented by independent distributed arrays
- Data not always in memory:
 - Inherent support for out-of-core operations, enabling large data-sets



Distributed ML algorithms for AF Classification



**Barcelona
Supercomputing
Center**

Centro Nacional de Supercomputación

Implementations

Dataset preparation

- **PCA: reduce the dimensionality** of the data before fitting. Loss of the 5% of the total information, preserving the 95% of the information contained in the features of the original dataset.
- The initial part of each PyCOMPSs execution in the results represents the PCA portion of the execution, not considered in execution time.

Machine Learning Classification

- Cascade Support Vector Machine, KNN, Random Forest

Neural Networks Classification

- CNN through PyCOMPSs + EDDL* (European Distributed Deep Learning) Library

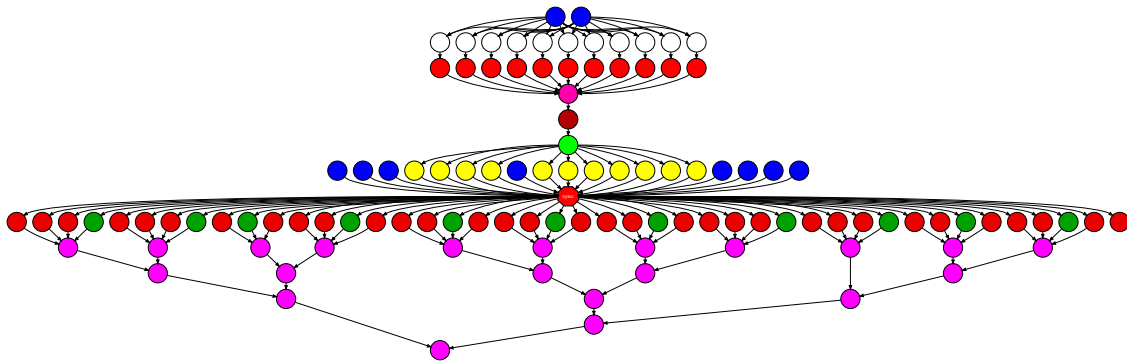


*J. Flich, C. Hernandez, E. Quiñones, and R. Paredes, "Distributed training on a highly heterogeneous hpc system," in Embedded Computer Systems: Architectures, Modeling, and Simulation, A. Orailoglu, M. Jung, and M. Reichenbach, Eds. Cham: Springer International Publishing, 2020, pp. 359–370.

Implementations

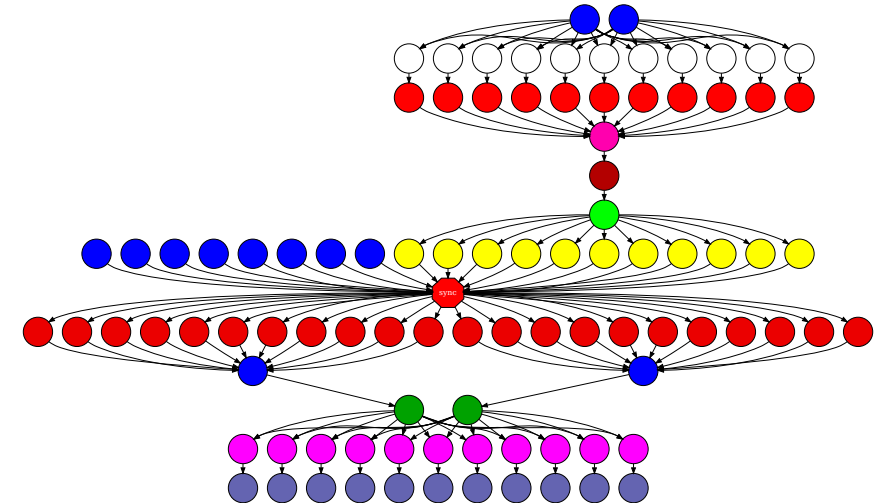
CSVM

- Dislib Blocks: 500x500
- 631 tasks managed by PyCOMPSs
- 6 tasks per node, 8 cores per task
- Parallelism: number of row blocks, limited by cascade



KNN

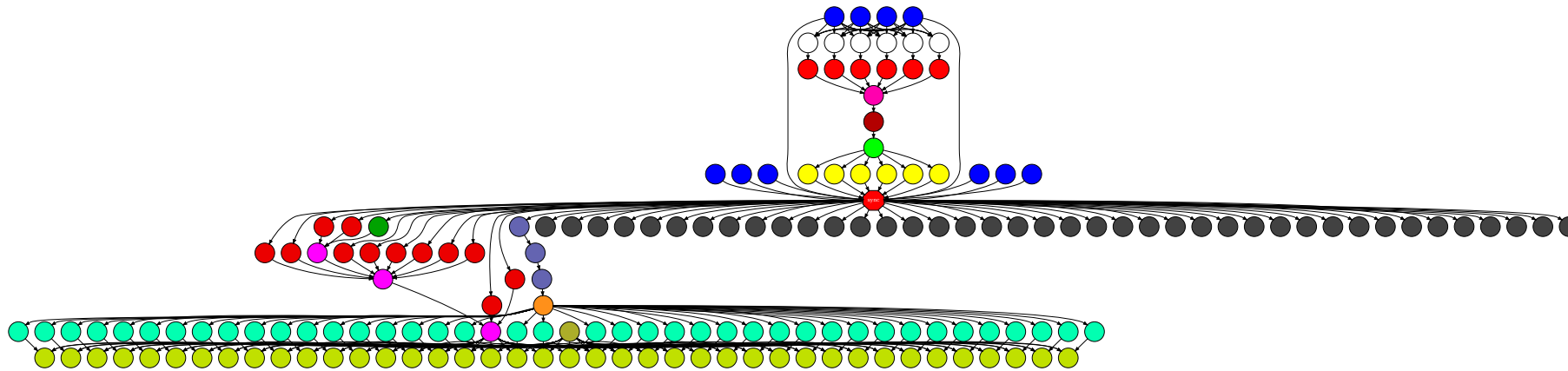
- Dislib blocks: 250x250
- K=5 neighbours
- 12 task per node, 4 cores per task
- Parallelism: number of row blocks of the dataset
- StandardScaler: adjust the values of all the features used to the same range



Implementations

Random Forest

- Number Estimators: 40
- Parallelism: is based on the number of estimators and the limit of the depth of the tree where the decisions are no longer computed in parallel



Evaluation



**Barcelona
Supercomputing
Center**

Centro Nacional de Supercomputación

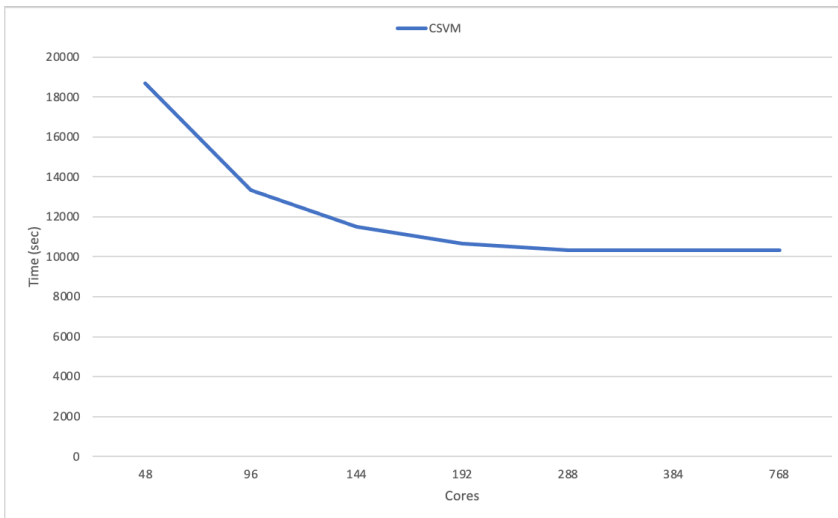
Evaluation

Each experiment is executed with a K-fold (K=5)

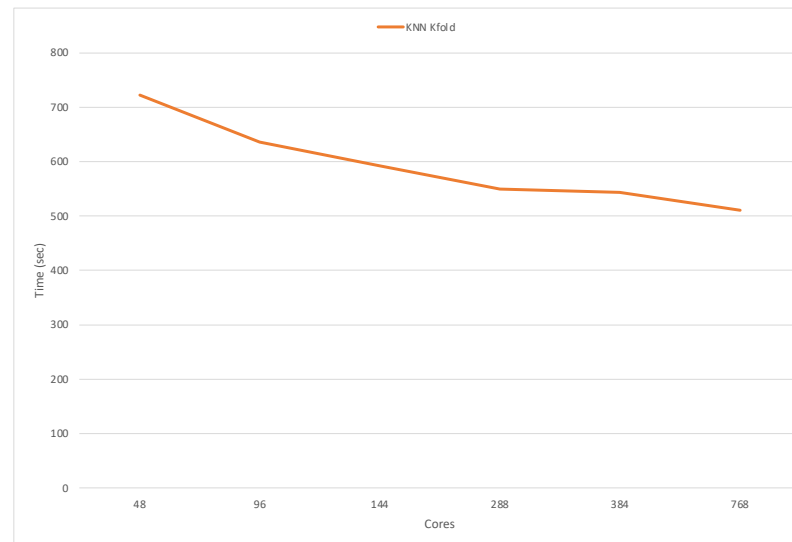
- CSVM scales up to 192 cores
- Random Forest scalability issues due to block size/number of nodes
- KNN quicker but yields to worst accuracy

Testbed: MareNostrum IV

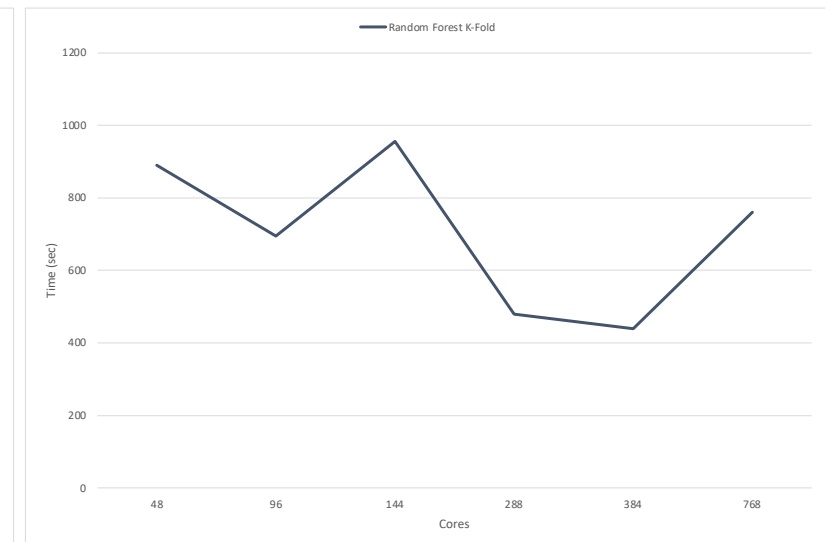
- General Purpose: 3456 nodes (48 servers of 72 nodes) with two 24-core Intel Xeon Platinum 8160 and 98 GB of main memory each.



Accuracy: 74,9%



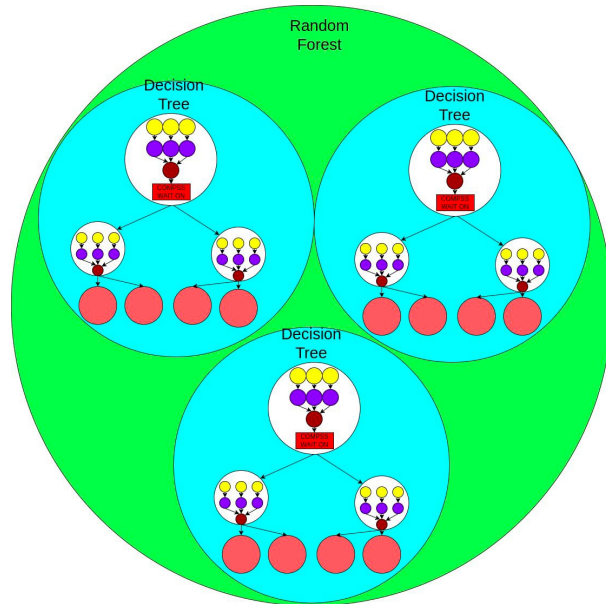
Accuracy: 52%



Accuracy: 86,8%

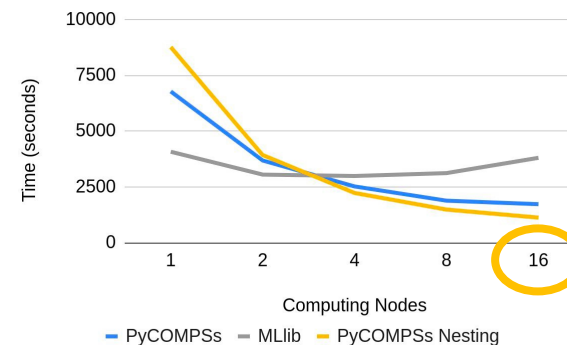
Dislib sample algorithm: Scalable Random Forest*

- New implementation of Random Forest to achieve better scalability
- Leveraging task nesting: generation of tasks inside other tasks and synchronizations within a task.



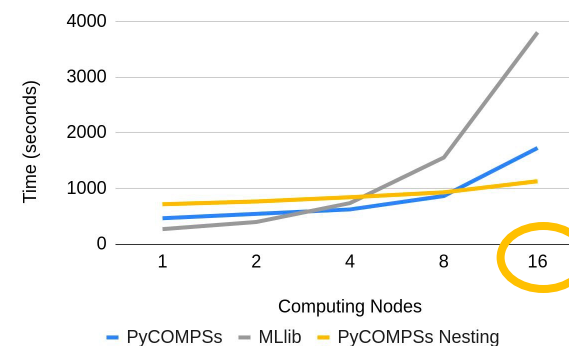
*Vázquez-Novoa, F. et al, "Scalable Random Forest with Data-Parallel Computing." In European Conference on Parallel Processing, 2023.

Strong Scaling



Higgs dataset:
11 million instances
28 attributes

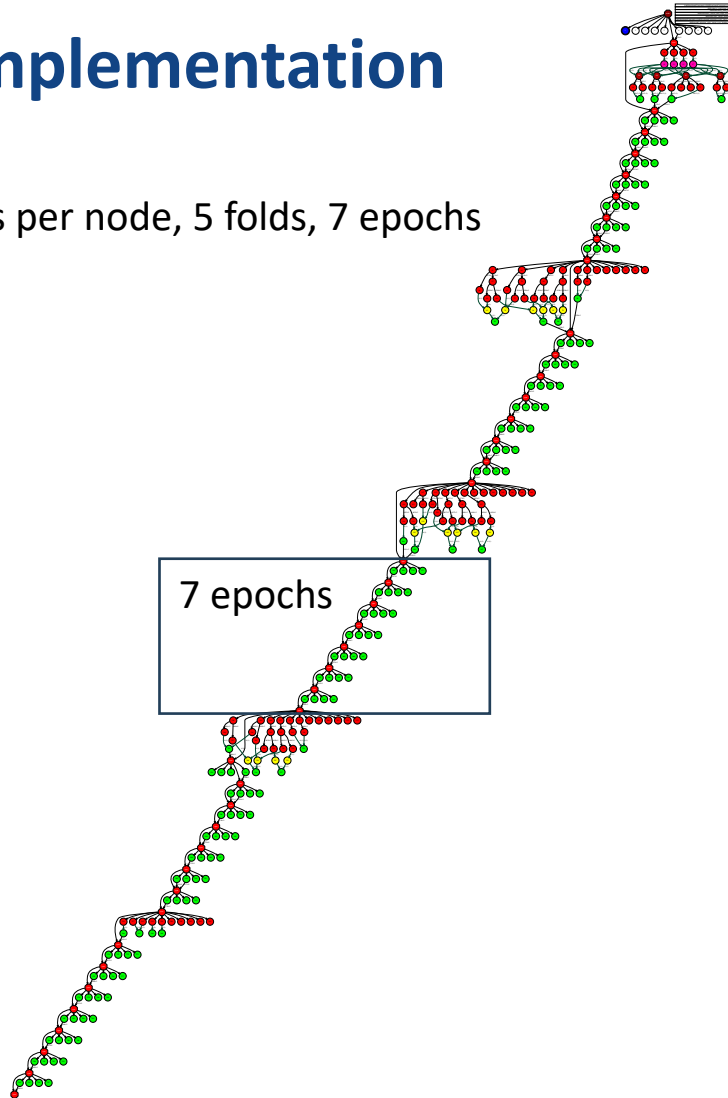
Weak Scaling



MareNostrum4
nodes: 48 cores each

NN Implementation

4 GPUs per node, 5 folds, 7 epochs



Integration of COMPSs with EDDL

2 approaches:

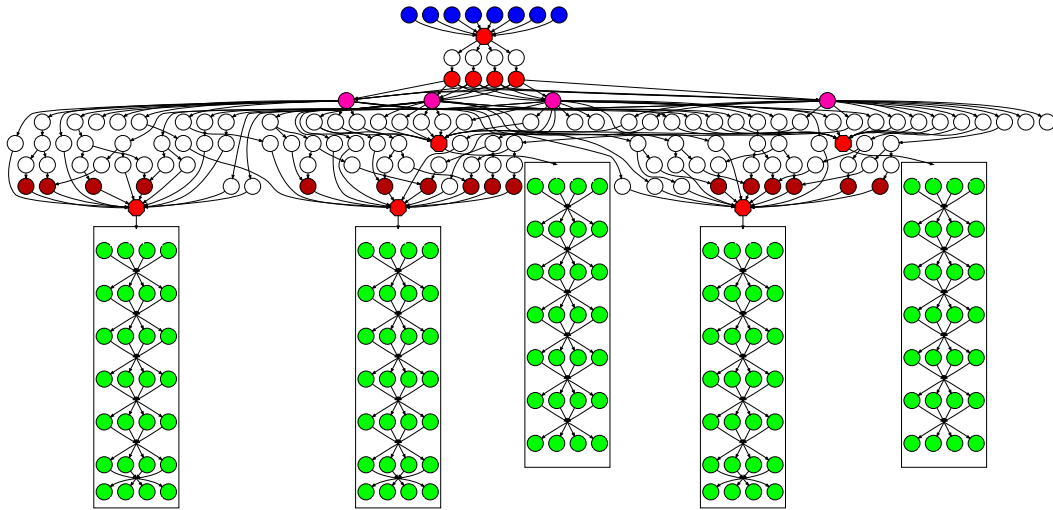
- 1) PyCOMPSs distributes the data between different nodes, with EDDL running on **4 GPUs inside each node**, being EDDL in charge of distributing the data between the different GPUs.
- 2) PyCOMPSs workers distribute the data between the different GPUs, and EDDL trains the model using only one GPU.

When an epoch ends, the weights of the neural network in each worker are retrieved and they are merged and used in the next epoch. After each epoch, a synchronization is required to retrieve the updated weights of the neural network in each worker: poor parallelism

Evaluation

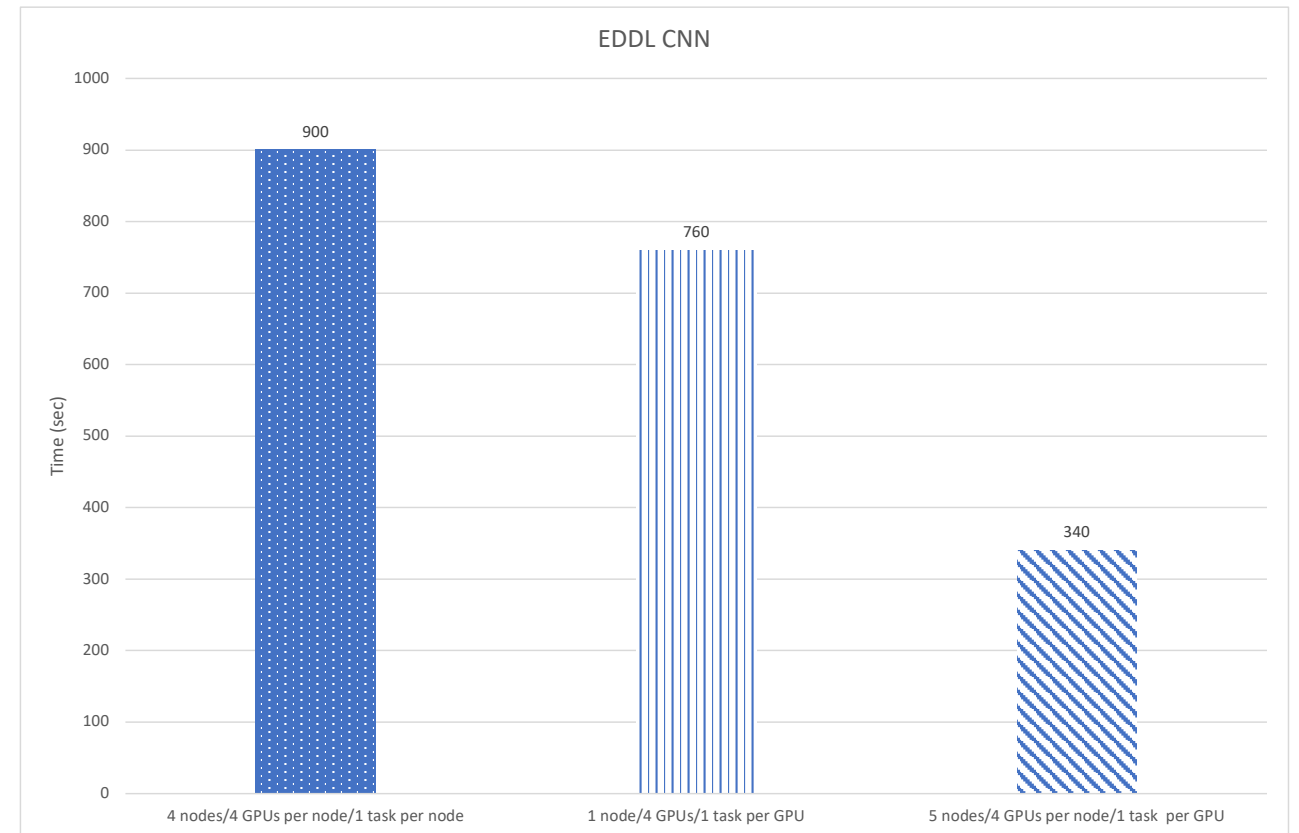
Nesting approach

5 nodes, 4 GPUs per node, 5 folds in parallel



Testbed: MareNostrum IV

- GPU: CTE-Power cluster, Power 9 GPU 52 compute nodes, each of them with 2 IBM Power9 8335-GTH @ 2.4GHz CPUs, 512GB of main memory and 4 GPU NVIDIA V100 (Volta) with 16GB HBM2.



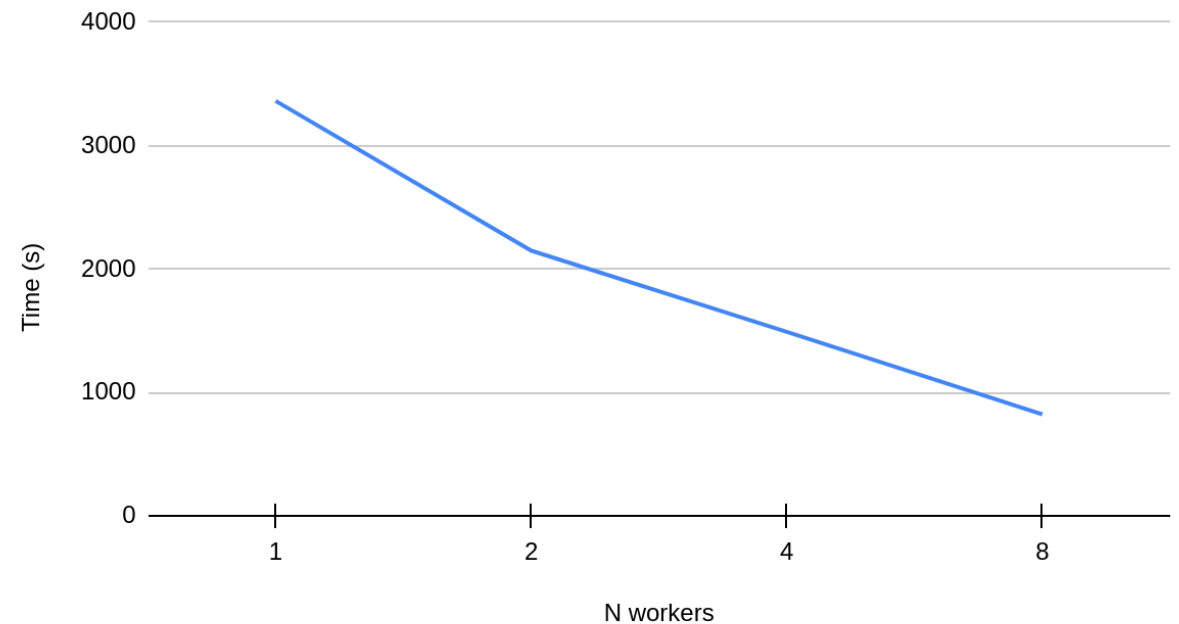
Accuracy: 90%

Future work

Distributed PyTorch with COMPSs

- Different training strategies for neural networks
- Each COMPSs task trains a copy of the neural network on a subset of the training data. Then, the global neural network is updated with the weights computed by all workers
- Each task is executed on a different GPU, parallelization of the execution in different GPUs inside the same task is not supported
- Some results obtained using Imagenette dataset (reduced version of Imagenet)
- The results shown follow a synchronous strategy, the weights computed by the different workers are synchronized at the end of each training epoch, and these weights are merged, updating the global neural network

Training time (15 epochs)



```
def main():
    net = ConvNeuralNetwork(num_classes)
    encaps_function = EncapsulatedFunctionsDistributedPytorch(num_workers)
    criterion = nn.CrossEntropyLoss
    optimizer = optim.SGD
    optimizer_parameters = {"lr": 0.01, "momentum": 0.5}
    x_train_tensor, y_train_tensor = load_dataset(...)
    x_test_tensor, y_test_tensor = load_dataset(...)
    encaps_function.build(net, optimizer, criterion, optimizer_parameters, num_gpu=4, num_nodes=1)
    trained_weights = encaps_function.fit_synchronous_shuffle_with_GPU(x_train_tensor,
        y_train_tensor, num_batches, num_epochs)
    net = assign_weights(net, trained_weights)
    for images, labels in zip(x_test_tensor, y_test_tensor):
        outputs = labels.to("cuda:0")
        predicted_outputs = net(images[None, :, :].float().to("cuda:0"))
        preds, predicted = torch.max(predicted_outputs, 1)
        outputs = torch.argmax(outputs)
        correct_predictions += (predicted.to("cuda:0") == outputs.to("cuda:0")).sum().item()
    print("Validation Accuracy is: " + str((100 * correct_predictions / y_test_tensor.shape[0])) + "%")
```

Reproducibility



- **Results replicated**

- Applications and data available in WorkflowHub
- <https://zenodo.org/records/13836996>



Tutorial: Optimising Workflow Lifecycle Management: Development, HPC-Ready Containers Deployment and Provenance

Time:

Monday, 18 November 2024
8:30am - 12pm EST

Location: [B209](#)

BoF: Reproducibility and Provenance for HPC and Distributed Environments

Time:

Tuesday, 19 November
2024 5:15pm - 6:45pm EST

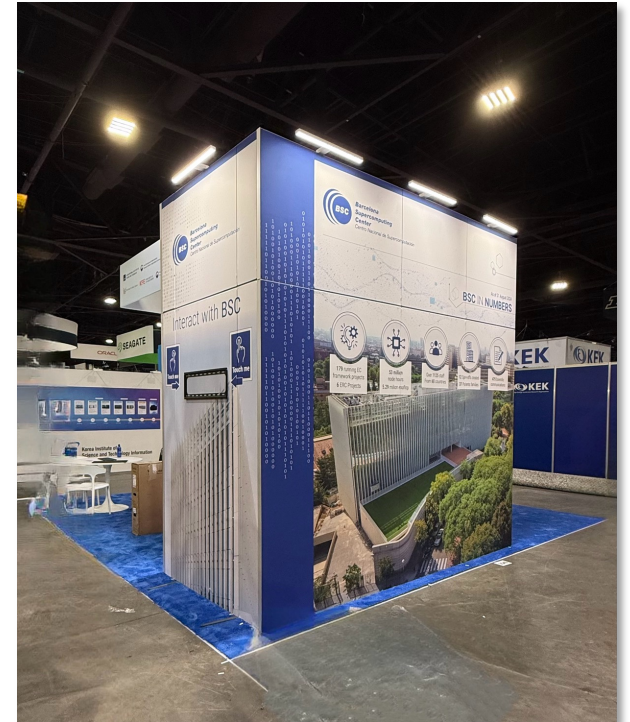
Location: [B201](#)

Conclusions

- **Unified programming model for the Continuum**
 - COMPSs provides **transparency** wrt the platform
 - Integration with heterogeneous HPC systems
 - Different approaches for model training with the same pattern (fit-predict)
 - Balance between accuracy/training time/resources
- **Future (and current) work**
 - Work with much bigger dataset
 - Improvement of scalability through **nesting** feature
 - Implementation of Federated Learning using distributed COMPSs agents
 - Extension of **NN** implementations

ACKs

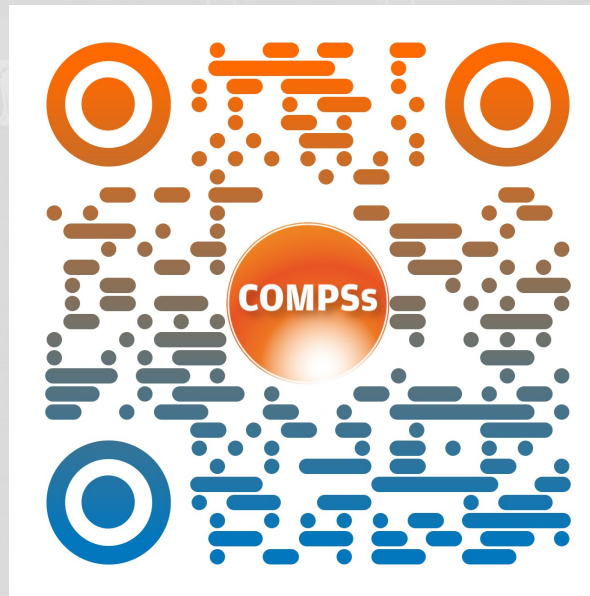
Visit us at BSC Booth #3549





**Barcelona
Supercomputing
Center**
Centro Nacional de Supercomputación

Thanks!



Daniele Lezzi
Senior Researcher at Barcelona Supercomputing Center



daniele.lezzi@bsc.es